Fine-Grained Computation Offload for Event-Driven Applications

Bojie Li
Pine AI

Abstract

Hardware accelerators—GPUs, TPUs, FPGAs, and increasingly *remote* services such as hardware security modules, post-quantum cryptography, and inference servers—now sit on the critical path of online serving. A request is received, pre-processed, handed to the accelerator, post-processed, and answered; while the accelerator works, the calling context stalls. For *fine-grained* offloads (microseconds to a few milliseconds), the classic responses—a context switch to another thread, or a busy-wait—either waste the CPU or add a wakeup latency comparable to the offload itself. The principled fix is to *overlap* the offload with other useful work, but this requires injecting concurrency into the application, and the real question is *how much the application must change*.

We study this question across the landscape of real, off-the-shelf servers and find a *spectrum*. At one extreme, an `LD_PRELOAD` M:N fiber runtime overlaps the offload with *zero* source modification—but only for thread-per-connection servers, and it hits three hard walls: engines that synchronize below libc, managed runtimes that keep thread identity in native thread-local storage, and workloads whose per-request CPU already exceeds the offload. At the other extreme, a *few lines* injected at the offload site overlap it through the application's own concurrency, and this works everywhere. Across ten production servers—Redis, nginx, memcached, Node.js, Python/asyncio, Apache, Go, PostgreSQL, MariaDB, and HAProxy—the change is **18–112 lines added and 0–1 lines modified**. Measuring on real hardware only—no emulated latencies—real overlap lands at 1.2–5× on stock servers (and 17.3× for the purpose-built transparent runtime), and we find it is bounded at *both* ends: by the accelerator's own throughput (a large GPU offload saturates the device's memory bandwidth, so overlap recovers its utilization rather than multiplying it) and by the server's own overlap capacity (a real latency-bound remote signer lifts the device ceiling but then hits the `asyncio`/GIL executor at 3.5×). The dramatic order-of-magnitude wins that an unbounded offload model would predict do not survive contact with real hardware. A simple model predicts both the speedup and the code cost from two properties: the server's concurrency model and the offload weight relative to per-request work. When the overlapped offload touches shared state, a transparent page-protection conflict detector preserves correctness, which we validate on a stock Redis server. The result is a practical recipe, and a predictive map, for hiding accelerator-offload latency in existing event-driven servers.

1 Introduction

Hardware accelerators are now a standard part of online serving. GPUs, TPUs, and FPGAs accelerate machine-learning inference, image processing, encryption, and compression, and an increasing share of "acceleration" is in fact a *remote* call—to a hardware security module that signs a token, to a post-quantum key-encapsulation service National Institute of Standards and Technology [2024], to a co-located inference server Crankshaw et al. [2017]. The shape of the application is remarkably uniform across these cases. A server receives a request from the network, does some pre-processing, invokes a computationally intensive routine, does some post-processing, and sends a response. When the intensive routine runs on an accelerator, it is replaced by an *offload*: a submission to the device and a wait for its completion. This raises a deceptively simple question that this paper is about: *what should the CPU do while it waits for the accelerator?*

The textbook answers are unsatisfying precisely in the regime that matters (Figure 1). The first is to *relinquish the CPU*: after submitting the offload, block, and let the operating system run another thread. But a context switch on Linux costs several microseconds, and a fine-grained offload—AES on a few kilobytes, a small inference, an HSM round-trip—also takes from a few to tens of microseconds. This is the "killer microsecond" regime Barroso et al. [2017]: too long to spin away, too short for the operating system to schedule around profitably. The switch overhead is then comparable to the work it overlaps: it wastes CPU and *adds* a thread-wakeup delay to the request's tail latency. The second answer is to *busy-wait* on the completion flag, which hides the wakeup delay but burns a core doing nothing. The third, and the only principled one, is to *do other useful work* on the same CPU while the offload is in flight—to *overlap* the offload with the processing of other requests. This is what high-performance systems already do by hand. The difficulty is that overlapping requires *concurrency* inside the application at the offload point, and existing servers express concurrency in many different ways. The central question of this paper is therefore not *whether* to overlap, but *how much an off-the-shelf server must be changed to do so*.

We answer this empirically, across the landscape of real servers, and the answer is a **spectrum** bounded by two extremes (Figure 8). At one extreme, overlap can be injected with *no source modification at all*. We build a user-space M:N fiber runtime, loaded by `LD_PRELOAD`, that interposes the standard threading and I/O symbols: it turns each connection-handling thread into a lightweight fiber, yields the fiber at every blocking point, and runs other fibers on the same core in the meantime. On a synchronous thread-per-connection handler it delivers large speedups—17.3× over a synchronous baseline for GPU AES—with the application binary unchanged. But transparency reaches only so far, and mapping exactly how far is itself a contribution. Running the runtime under stock binaries surfaces a sequence of obstacles that any threading-interposing tool must solve (most notably a glibc condition-variable symbol-versioning hazard that silently corrupts some binaries), and, beyond those, *three architectural walls* that no engineering removes: storage engines that synchronize with raw `futex` system calls *below* the libc interface, managed runtimes that store thread identity in a native thread-local slot the loader cannot virtualize, and workloads whose per-request CPU already exceeds the offload so that there is no idle time to reclaim. Crucially, the runtime engages only on *thread-per-connection* servers: an event-driven server has a single event loop and no per-connection thread to fiberize, so the very class of applications with the *most* to gain from overlap—one synchronous offload stalls the *entire* server—is the class transparency cannot help.

At the other extreme, overlap can be injected with *a few lines*. The recipe is uniform: replace the synchronous offload at its call site with an asynchronous one that hands the work to the application's *own* concurrency mechanism—a module's background thread pool, an event loop's deferred-response primitive, a language runtime's tasks—and answer the request on completion. The edit is small because every modern server already contains machinery to suspend and resume work; one only has to route the offload through it. We instantiate this recipe on **ten** off-the-shelf servers spanning every concurrency model—Redis, nginx, memcached, Node.js, Python/asyncio, Apache, Go, PostgreSQL, MariaDB, and HAProxy—and in each case the change is **18 to 112 lines added, and zero or one existing line modified**. Measured on real hardware throughout—no emulated latencies—the edit recovers 1.2–5× with no failed requests, bounded at both ends by the accelerator's throughput and by how much concurrency the server can itself express (§7).

Behind these numbers is a simple predictive model, and extracting it is the paper's main intellectual contribution. The speedup *and* the code cost of overlapping an offload are determined by two properties of the server: its *concurrency model* and the *weight of the offload relative to the per-request CPU work*. The concurrency model places a server in one of a few regimes, and the regime predicts both how much overlap a minimal edit buys and how many lines it takes (which is dominated by whether the server already exposes an asynchronous, deferred-response primitive). The offload weight predicts whether overlap helps at all: when per-request CPU dominates, even a perfectly overlapped offload changes little. This model explains the full range of our measurements, including the cases where overlap does *not* help, and it tells an operator in advance which servers to touch and what to expect.

Overlap introduces one correctness hazard, and we address it. When the post-processing after an offload mutates shared state—a global counter, a cache, a key in a key-value store—then overlapping two requests can reorder their read-modify-write sequences and corrupt that state. We provide a transparent conflict detector, based on page protection and version snapshots, that flags exactly these conflicts and serializes only the conflicting work; it applies at both ends of the spectrum and needs no application annotation.

In summary, this paper makes the following contributions:

- **A predictive model** of fine-grained offload overlap in existing servers: speedup and code cost as a function of the server's concurrency model and the offload's weight relative to per-request work, validated across ten real servers (§5).
- **The transparent boundary, characterized**: a zero-modification M:N fiber runtime, the obstacle taxonomy it must overcome (including a reusable glibc condition-variable versioning bug), three architectural walls where transparency provably stops, and a classifier that predicts whether a binary is fiberizable (§3).
- **A minimal-edit recipe and a cross-landscape study**: one offload-overlap pattern on ten off-the-shelf servers with 18–112 lines added and 0–1 lines modified, evaluated on real hardware (1.2–5×)—and the finding that the win is bounded at both ends, by the accelerator's throughput (a large GPU offload saturates the device's bandwidth) and by the server's own overlap capacity (a real latency-bound remote signer tops out at 3.5× behind a GIL-bound executor) (§4, §7).
- **A transparent conflict detector** that keeps overlapped offload correct when it mutates shared state, with no application changes (§6).

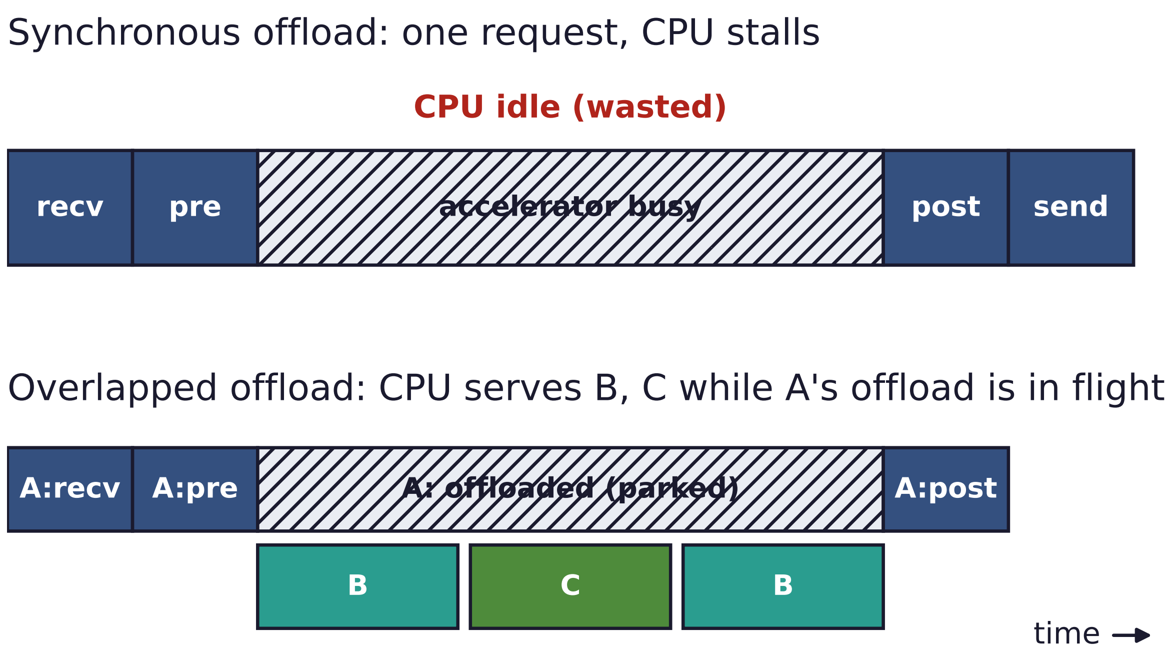

Figure 1. The problem. A serving request is *receive* → *pre-process* → *offload* → *post-process* → *send*. Top: with a synchronous offload, the CPU stalls while the accelerator works. For fine-grained offloads (microseconds to a few milliseconds), a context switch to fill the gap costs as much as the gap itself, and busy-waiting wastes the core. Bottom: *overlap* fills the gap with other requests' processing. The question of this paper is how much an existing server must change to do this.

2 Background and Motivation

Accelerators and the fine-grained regime. An accelerator turns a CPU-bound routine into a device operation: the CPU prepares an input buffer, submits it (a kernel launch, a DMA, or a network send), and later collects the result. The *latency* of this round-trip spans a wide range. Bulk AES or a small CUDA kernel on a few kilobytes completes in tens of microseconds; a compression card or a small on-GPU inference in tens to hundreds of microseconds; a hardware security module signature, a post-quantum key-encapsulation, or a remote inference call in milliseconds. What unites them (Figure 2) is that they are *fine-grained* relative to operating-system scheduling: the offload finishes on a timescale comparable to, or only modestly larger than, a context switch. In this regime the classic ways of waiting are wasteful, as Figure 1 illustrates—blocking pays a switch and a wakeup that rival the offload, and busy-waiting pays a whole core. Overlapping the offload with other work is the only approach that neither wastes the CPU nor inflates latency, and it is the approach this paper makes easy to adopt.

How servers express concurrency. "Other work to overlap with" means *concurrent requests*, and a server's ability to overlap an offload is determined by how it runs concurrent requests in the first place. We find it useful to group servers into a few models, which recur throughout the paper. A *single-event-loop* server (e.g., Redis, Node.js, a Python `asyncio` service) multiplexes all connections on one thread that loops over ready events; an *event-loop-with-pool* server (nginx, memcached) runs a few such loops plus a worker pool for blocking work; a *thread- or goroutine-pool* server (Apache, Go) dispatches each request to a pooled worker or a runtime-scheduled task; a *proxy* (HAProxy) forwards each request through a native offload engine rather than

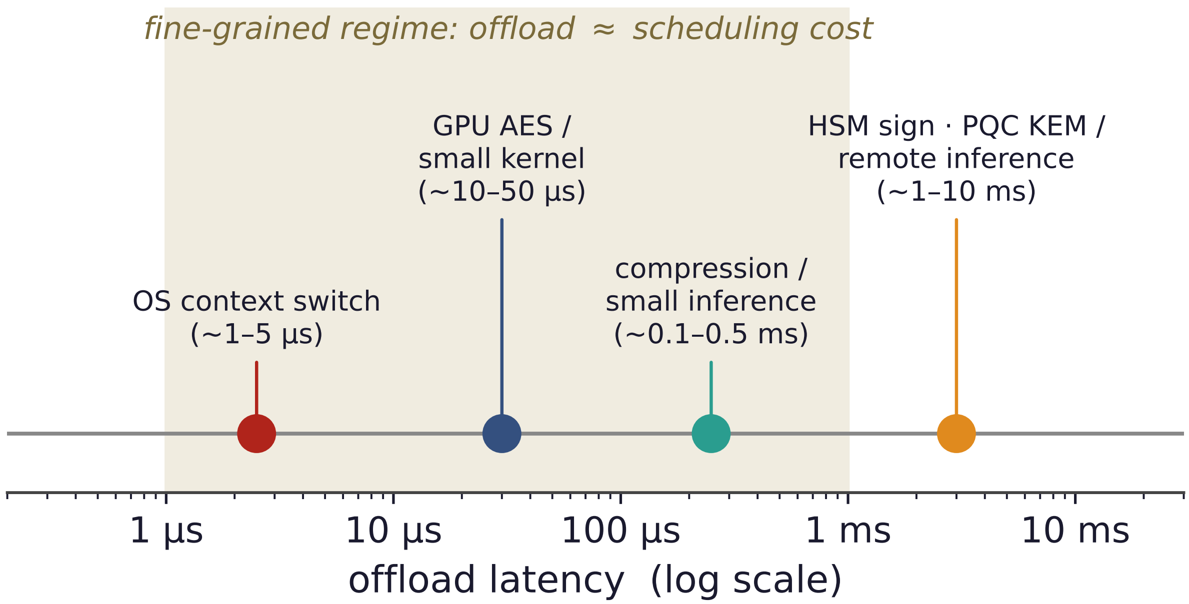

Figure 2. The fine-grained regime. Accelerator offloads span microseconds to milliseconds. In the shaded band, the offload latency is comparable to an OS context switch, so blocking pays a switch and a wakeup that rival the work itself, and busy-waiting wastes a core. This is exactly where overlapping the offload with other requests is the right answer.

running application logic itself; and a *process- or thread-per-connection* server (PostgreSQL, MariaDB) gives each connection its own backend. These models differ sharply in what a synchronous offload *costs* and in how overlap must be injected—differences this paper turns into a predictive model (§5).

The motivating catastrophe. The starkest case is a single event loop. Because one thread handles every connection, a synchronous offload on that thread blocks *all* connections for its full duration; the server's throughput collapses to roughly one request per offload latency, no matter how many clients are waiting. The hardware is almost entirely idle, yet the server is saturated—a pathology of the *structure*, not of capacity. A few lines that move the offload off the loop restore full concurrency and recover more than an order of magnitude (§7). The event-driven servers that dominate modern serving are thus the ones with the most to gain from overlap—and, as we show next, the ones a transparent runtime cannot help.

3 Transparent Overlap: The Zero-Edit Extreme

We begin at the zero-modification end of the spectrum. The goal is the most convenient form of overlap imaginable: take an *unmodified* server binary and make it overlap its offloads, with no recompilation and no source access. The result is an existence proof that the mechanism works, and— more importantly for the rest of the paper—a precise map of how far transparency can reach.

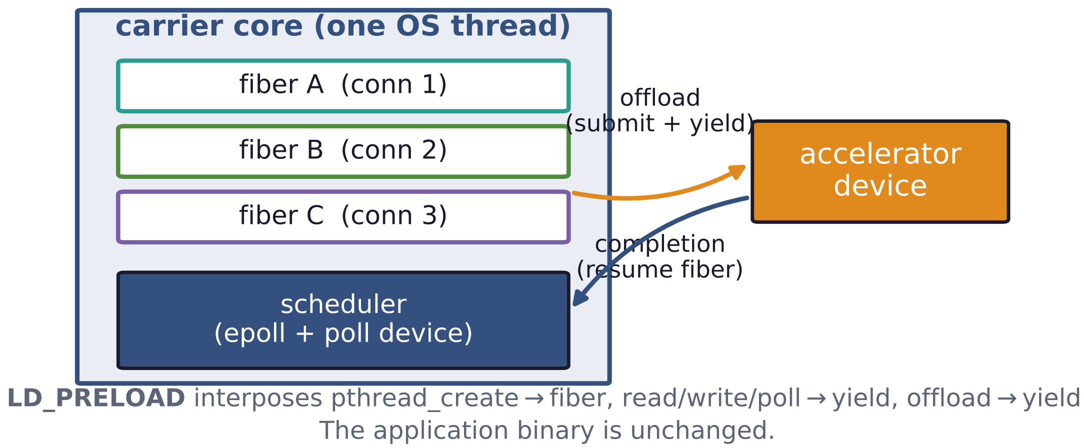

Figure 3. The transparent runtime. An `LD_PRELOAD` library turns each connection thread into a fiber on one carrier core; a fiber yields at its offload (and at socket I/O), and the scheduler runs another fiber until the device completes. The application binary is unchanged. On a synchronous thread-per-connection handler with a real GPU this overlaps 64 connections' offloads and delivers 17.3× over a blocking baseline.

3.1 An `LD_PRELOAD` M:N fiber runtime

Our runtime (Figure 3) is a shared library loaded ahead of the application by `LD_PRELOAD`. It interposes the standard threading and I/O symbols. When the application creates a connection-handling thread, the runtime instead spawns a *fiber*—a user-level thread with its own small stack and a hand-written, register-only context switch of tens of nanoseconds, some twenty times cheaper than a kernel switch Li et al. [2023]. All fibers run on a single *carrier* OS thread. When a fiber would block—on a socket read or write, or on the offload—the runtime saves its context and switches to another runnable fiber; a scheduler on the carrier waits for I/O readiness and offload completions and resumes the corresponding fiber. The application's handler keeps its plain, synchronous shape (read a request, call the offload, write a reply); it never learns that its "thread" is a fiber or that its offload was overlapped with its neighbors'. Per-fiber state that the C library treats as thread-local, such as `errno` and the OpenSSL error queue, is saved and restored at every switch so that fibers do not see each other's transient state.

On a synchronous thread-per-connection handler this works well. With a real GPU performing AES, the runtime overlaps 64 connections' offloads on one carrier core and reaches 17.3× the throughput of the same binary blocking on each offload, with results verified bit-for-bit; a DNN-inference server shows 11.8×. These confirm the mechanism. The rest of this section is about its limits, which turn out to be the more useful contribution.

3.2 What it takes to run real binaries

Interposing the threading layer of a stock binary is harder than it sounds, and the obstacles are worth recording because any such tool faces them. The sharpest is a *symbol-versioning* hazard Drepper [2011]. The glibc condition-variable functions carry two incompatible ABIs

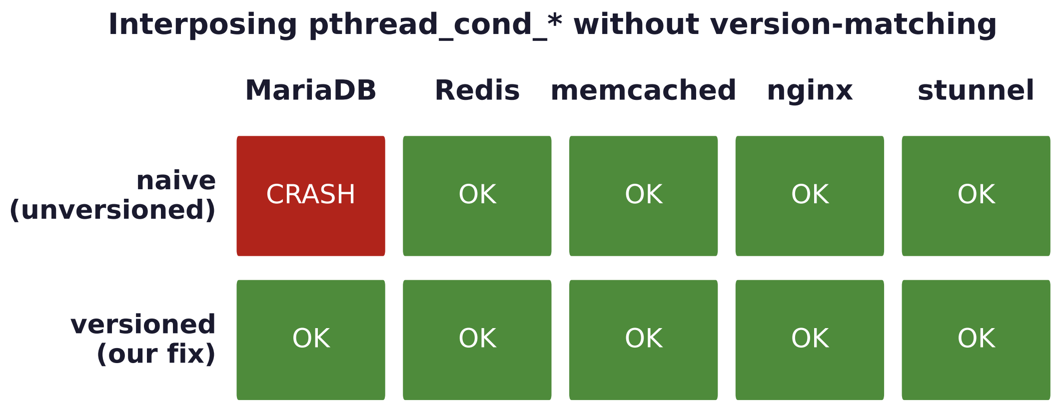

Figure 4. A reusable interposition hazard. The glibc condition-variable functions carry two incompatible ABIs under one name; a naive unversioned interposer breaks the version-matched relocation and corrupts *some* binaries (MariaDB crashes at startup) while silently sparing others. Version-matching the interposed symbols fixes all of them. Any threading-interposing tool must do this.

under the same names (`GLIBC_2.2.5` and `GLIBC_2.3.2`); a naive interposer that defines an *unversioned* `pthread_cond_signal` silently breaks the dynamic linker's version-matched relocation and corrupts the application. MariaDB crashes at startup with a wild pointer; the fix is to export the interposed symbols at their exact version and resolve the real ones with `dlvsym`. The same defect is latent in any threading-interposing tool, and we found it crashes one of five tested server binaries while silently sparing the rest (Figure 4)—a reusable practitioner lesson. Other obstacles, each resolved once: the server's I/O may use `recv`/`send`/ `poll` rather than `read`/`write`; `pthread_self` must return the real OS thread identity so that the C library's stack-bounds logic works; a daemon's `fork` drops the carrier thread, which must be re-established; and pinning the carrier to a real-time priority can invert priorities against a lock holder.

3.3 Three walls where transparency stops

Beyond engineering obstacles lie three *walls* that no amount of interposition removes (Figure 5). Each is a place where the application's behavior escapes the layer a preloaded library can intercept.

Wall 1—synchronization below libc. A storage engine such as InnoDB does its internal locking with the `futex` system call Franke et al. [2002] issued *directly*, bypassing the `pthread` layer entirely. A userspace runtime interposes library *symbols*, not system calls; a fiber that blocks in a raw `futex` therefore stalls the entire carrier, and under contention the whole server deadlocks. We confirmed this with a live backtrace of the frozen carrier inside InnoDB's synchronization code.

Wall 2—thread identity in native TLS. A managed runtime such as the JVM stores "the current thread" in a native thread-local slot read directly from a CPU register, not through any library call. Fibers that share one carrier OS thread cannot be given distinct values of that slot by symbol interposition, so after a fiber switch the runtime reads the *wrong* thread object and segfaults. We observe exactly this—a crash dereferencing a stale `Thread*` the moment two fiberized JVM requests interleave—and the same mechanism applies to Go and .NET.

Wall 3—no idle CPU to reclaim. Overlap only helps if the CPU is idle during the offload.

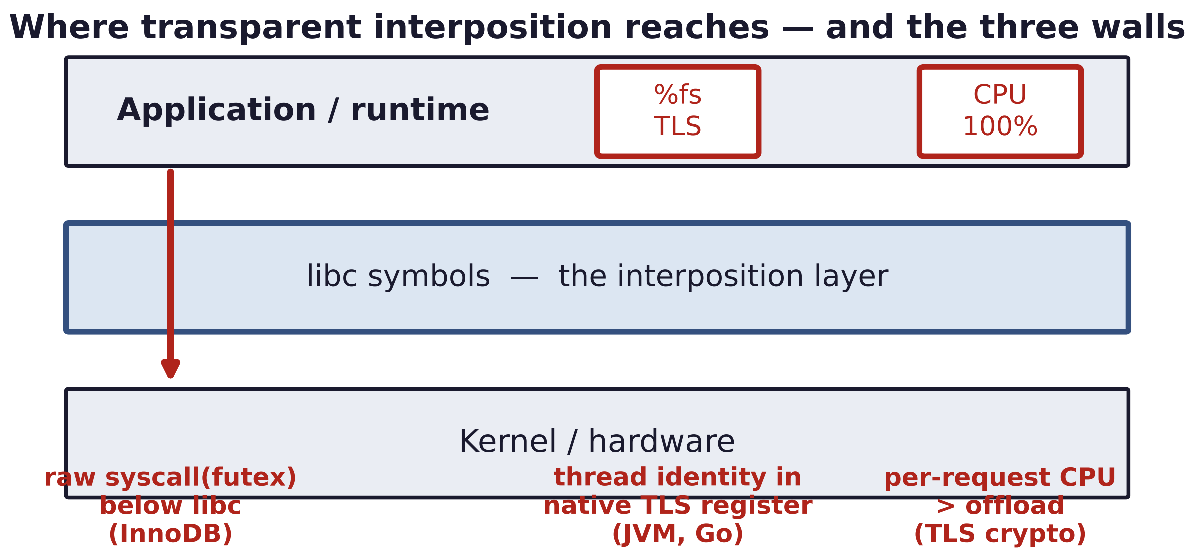

Figure 5. The three walls. Transparent interposition virtualizes the libc symbol layer (read/write, the `pthread` primitives, `errno`). It cannot reach three things: synchronization issued as a *raw* system call below libc (InnoDB's `futex`); thread identity held in a native TLS register (the JVM, Go); and workloads where per-request CPU already exceeds the offload, leaving no idle time. These bound where zero-edit overlap is possible.

A TLS terminator that is already thread-per-connection and spends real CPU on per-request crypto has no such idle time: the OS already overlaps its offloads across threads, while the single carrier serializes the real crypto and pays an interposition tax on every yield. The result is 2–3× *slower* than native on the same core.

3.4 A classifier for fiberizability

The walls are not arbitrary; they are predictable from a binary's behavior. We profile the *per-request blocking syscalls* a server makes under load. Event-driven servers do their I/O with `epoll` and `read` and have no per-connection thread to fiberize: the runtime loads safely but never engages. Thread-per-connection servers whose blocking is all at the libc layer have *near-zero* per-request `futex` traffic and are fiberizable. Engines that synchronize below libc have `futex` traffic that dominates their profile and use kernel-bypass I/O such as `io_uring` Axboe [2019]—the wall. Figure 6 makes this concrete: InnoDB's per-request `futex` density is roughly two orders of magnitude above a fiberizable server's, a cheap static-plus-dynamic check that tells an operator in advance whether a binary can be fiberized.

The verdict frames the rest of the paper. Transparency serves a narrow niche—thread-per-connection servers with light per-request CPU and a libc-level offload—and, by construction, it cannot help the event-driven servers that have the most to gain. For everyone else, a few lines suffice.

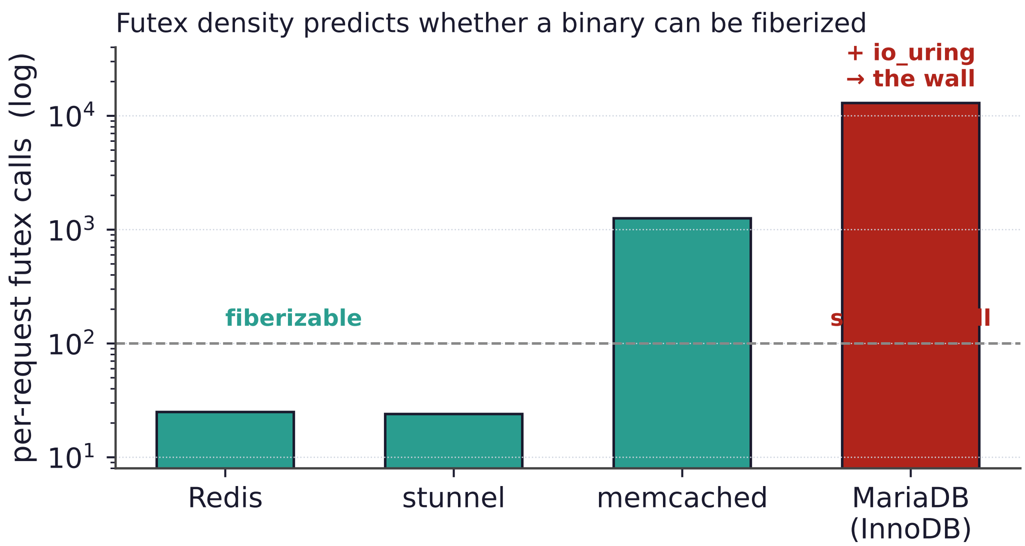

Figure 6. Predicting fiberizability. Per-request `futex` calls separate the near-zero libc-level servers (whose blocking the runtime can interpose) from engines that synchronize below libc (InnoDB, ~13,000, plus `io_uring`)—the wall of Figure 5. The same profile flags event-driven servers as "loads safely, no overlap."

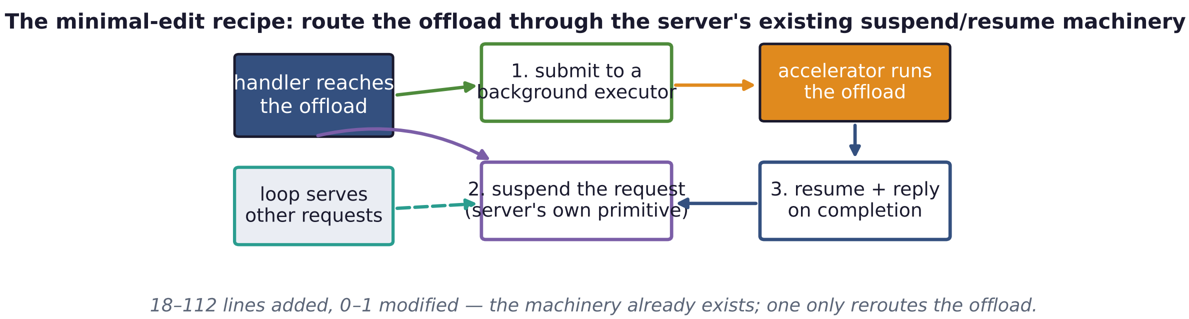

Figure 7. The minimal-edit recipe. At the offload call site, submit the work to a background executor, suspend the request with the server's own deferred-response primitive, and resume it to reply on completion; meanwhile the server's loop serves other requests. The edit is small because the suspend/resume machinery already exists—one only reroutes the offload through it.

4 Minimal-Edit Overlap

If a server cannot be fiberized from the outside, it can almost always be overlapped from a small edit inside. The reason is that every modern server already contains the machinery to *suspend* a request and *resume* it later—that machinery is what lets it serve many connections at once. To overlap an offload, one only has to route it through that machinery instead of blocking on it. The recipe is uniform across servers:

1. at the offload call site, *submit* the work to a background executor instead of waiting;
2. *suspend* the current request using the server's own deferred-response primitive;
3. *resume* it and send the reply when the offload completes.

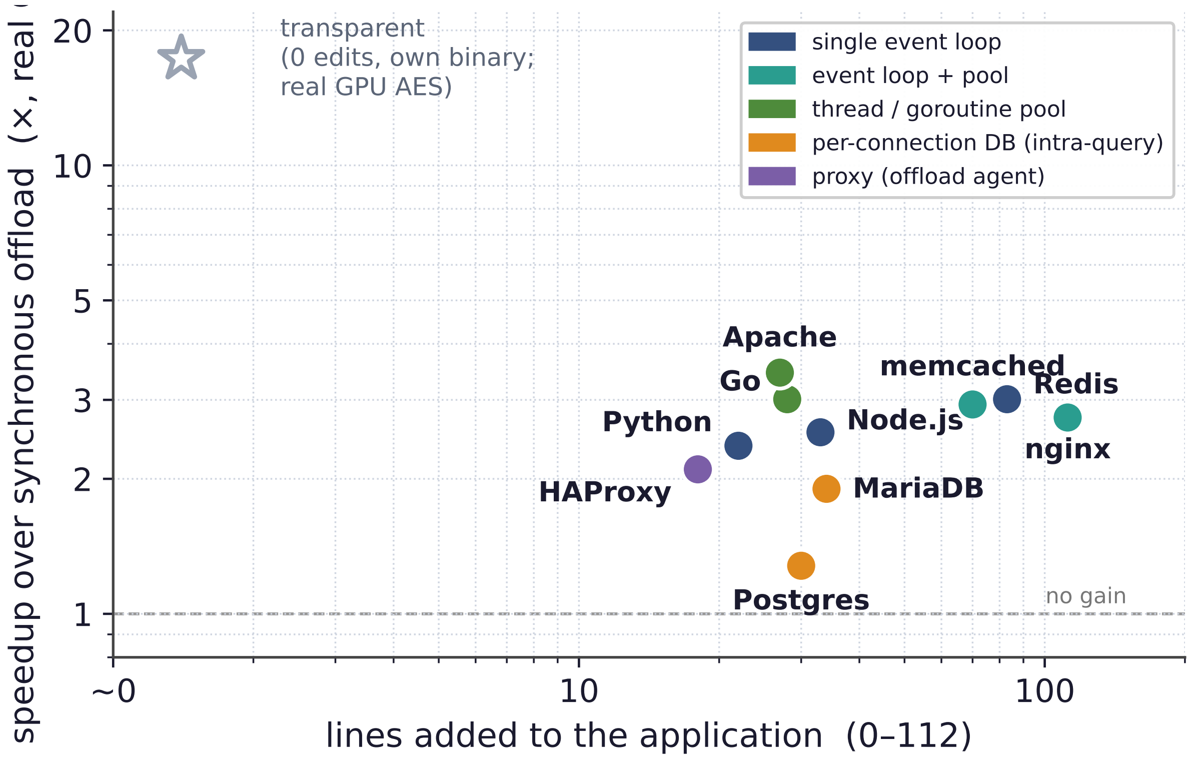

Figure 8. The spectrum (the paper in one plot), all on a *real GPU* with a 1 MiB AES offload, measured on an idle GPU. Each point is an off-the-shelf server; the x-axis is the lines of integration code, the y-axis the real-GPU speedup from overlapping the offload. The few-line edit clusters at 2–3.5×—a real bandwidth-bound GPU offload lets overlap fill the device but not exceed its throughput. The databases sit lower (intra-query pipelining only, as their backends already overlap across connections); HAProxy's proxy reaches 2.1× with a C offload agent (a GIL-bound Python agent gets only 0.9×). The zero-line transparent runtime reaches 17.3× in its niche on a small launch-bound offload (walls on third-party binaries). Color encodes the concurrency model (§5).

We distinguish two quantities: *lines added* (the new integration code) and *existing lines modified* (edits to code already in the server). The second is the invasive, maintenance-costly part; zero is ideal and is achievable wherever the server exposes a plugin or extension interface. We instantiate the recipe on ten servers spanning every concurrency model, and in every case the change is **18–112 lines added and 0–1 lines modified** (Figure 8).

The integration point follows the concurrency model. Servers with a *plugin or extension API* take the recipe with zero core edits: a Redis Redis Ltd. [2024] module blocks the client and runs the offload on a worker pool; an nginx F5/NGINX [2024] add-on posts the offload to nginx's thread pool and completes the request from the callback; an Apache Apache Software Foundation [2024] module is a content handler whose worker pool overlaps offloads for free; a PostgreSQL PostgreSQL Global Development Group [2024] extension and a MariaDB MariaDB Foundation [2024] user-defined function pipeline a query's offloads; a Node.js OpenJS Foundation [2024] N-API addon uses the libuv libuv contributors [2024] asynchronous-work API; and a HAProxy HAProxy Technologies [2024a] deployment streams each request to an external offload *agent* over the proxy's native Stream Processing Offload Engine HAProxy Technologies [2024b]. Servers backed by a *language runtime* need *no asynchronous code at all*: a Go The Go Authors [2024] handler that makes a plain blocking `cgo` offload is overlapped automatically because the goroutine scheduler runs other requests on other OS threads, and a Python `asyncio` Python Software Foundation [2024] service offloads through `run_in_executor` with a one-line change and no native build. The lone case that touches existing code is memcached memcached [2024], which has no asynchronous-command interface; we add a connection state and the park/resume transition into its state machine—70 lines added, one modified. memcached is thus the exception that proves the rule: the line count is dominated by whether the server already exposes a deferred-response primitive, a point we formalize next.

5 A Predictive Model of Offload Overlap

The measurements above are not a list of unrelated wins; they follow a pattern. Two properties of a server predict both *how much* overlap helps and *how many lines* it takes: the server's *concurrency model* and the offload's *weight relative to per-request CPU work*.

Concurrency model predicts the regime. Figure 9 lays out the regimes. A *single event loop* is the extreme case: a synchronous offload stalls the whole server, so the win is *dramatic*—the few-line fix turns a fully blocked loop back into full concurrency—and grows with offload weight up to the accelerator's throughput ceiling. An *event loop with a worker pool* stalls only one loop of several, so the win is *large* but bounded by the number of loops. A *thread or goroutine pool* overlaps offloads *automatically*, with zero asynchronous code, up to the pool size—the runtime is already doing the work. A *proxy* with a native offload engine treats async offload as a first-class, configuration-only feature. Finally, a *process- or thread-per-connection* server already overlaps *across* connections for free (the OS runs the backends concurrently), so the only edit-addressable win is *within* a single query, by pipelining a serial loop of offloads.

Code cost follows the same axis. The number of lines is governed by whether the server exposes an asynchronous, deferred-response primitive. Plugin and extension interfaces (Redis, nginx, Apache, the databases) and language runtimes (Go, Python) provide one, so the integration is purely additive—zero existing lines change. A server without such a primitive forces the developer to add the suspend/resume transition to its request state machine by hand; memcached is our only such case, and it is the only one that modifies an existing line.

Offload weight predicts whether overlap helps at all. The second axis is independent of the server (Figure 10). Overlap reclaims the time spent waiting for the offload; if the per-request CPU work is already comparable to the offload, there is little to reclaim. Sweeping a *real* GPU AES offload by block size on a single-event-loop server, a *light* block (4 KiB, 46 µs—on the order of the runtime's own per-request overhead) yields essentially no speedup (1.24×), and the speedup grows monotonically with offload weight to 5.41× at 8 MiB (2.3 ms), where it approaches the GPU's bandwidth (full per-size values in Appendix A). The rule is simple and it bounds applicability: overlap pays exactly when the offload outweighs the per-request CPU work *and* the accelerator is not already saturated. It also explains Wall 3 of §3, which is the same condition seen from the transparent side.

Together the two axes form a map: given a server's concurrency model and an offload's latency, one can predict before writing any code whether to bother, how large the win will be,

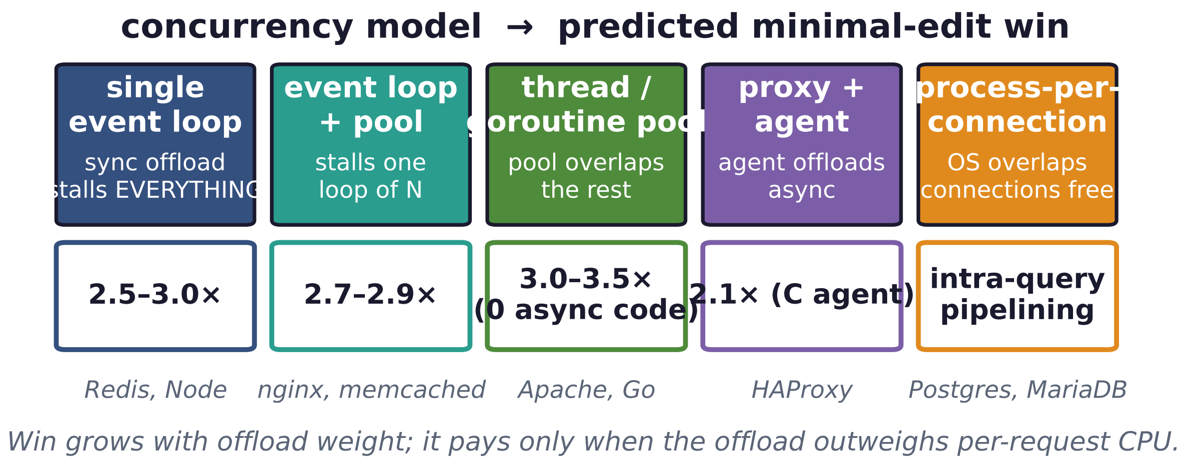

Figure 9. The predictive model, part one: the concurrency model determines what a synchronous offload costs and therefore how much a minimal edit recovers. The win ranges from *dramatic* (a blocked single loop) to *automatic* (a pool that overlaps for free) to *intra-query* (databases that already overlap across connections). Example servers are shown under each regime.

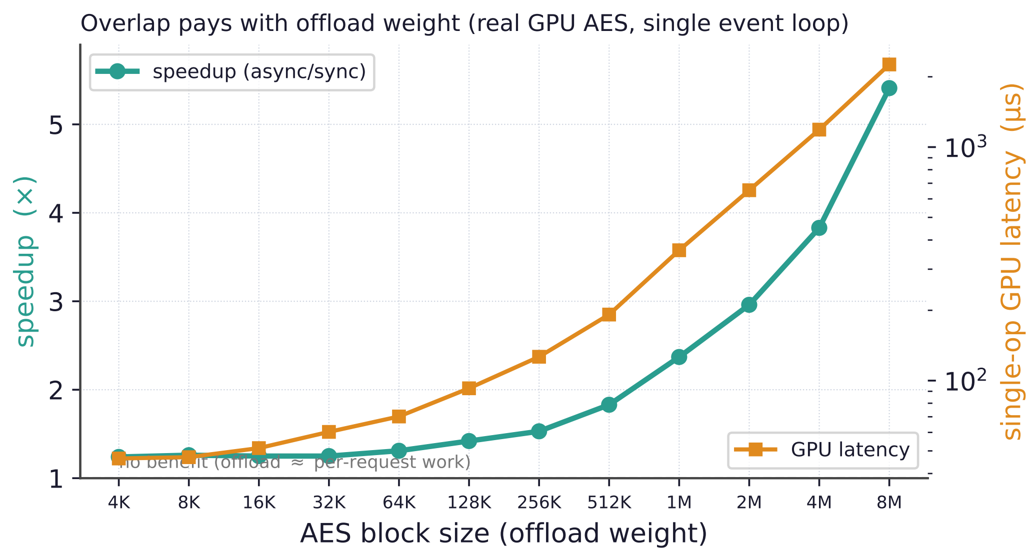

Figure 10. The predictive model, part two (measured, real GPU, idle): overlap pays only when the offload outweighs per-request CPU work. Sweeping a real GPU AES offload by block size (consecutive powers of two, 4 KiB–8 MiB) on a single-event-loop server, the same edit gives no benefit for a light block (1.24× at 4 KiB, 46 µs) and rises to 5.41× at 8 MiB (2.3 ms), approaching the GPU's bandwidth. Right axis: measured single-op GPU latency. This condition also explains the transparent runtime's third wall.

and roughly how many lines it will take.

6 Correctness: Overlapping Stateful Offloads

Overlap reorders work, and reordering is dangerous when the work touches shared state. If the post-processing after an offload performs a read-modify-write on a global—a request counter, a cache entry, a key in a store—then overlapping two requests can interleave their read-modify-write sequences and lose updates. We make this concrete on a *stock Redis server*: a module command reads a key, runs a real GPU offload, and writes the incremented value back, so the read-modify-write spans the offload. Driven by 30,000 overlapped requests across a small key space, unprotected overlap silently loses 23,450 updates and corrupts the result (Figure 11, left). The danger is subtle precisely because the offload makes the read-modify-write window long: the wider the latency overlap hides, the wider the race it opens.

Two mechanisms restore correctness without changing the application. First, *per-connection ordering* comes for free: because the runtime assigns one fiber per connection, a connection's dependent requests are serialized automatically, so pipelined order-dependent traffic is always correct. Second, for state shared *across* connections we provide a transparent *conflict detector*. It write-protects the application's shared-state pages, snapshots a version clock when a handler parks at its offload, and treats a write fault that lands on a page modified during the park as a conflict; under enforcement it holds a lightweight handler lock from the read to the write across the offload, so that only *conflicting* handlers serialize while everyone else stays overlapped. On the stock-Redis workload the detector catches essentially every conflict (26,185 flagged against 23,450 lost) and, under enforcement, reduces lost updates to *zero*. Crucially, it preserves overlap where a lock would destroy it: a coarse lock serializes the offloads it is meant to protect (13,812 req/s), whereas the detector keeps non-conflicting offloads in flight (24,691 req/s) and so runs 1.8× faster while staying correct (Figure 11, right). The detector is a property of the overlap mechanism, not of any one integration, so it applies equally to transparent fiberization and to the minimal-edit recipe.

7 Evaluation

Setup. All offloads are real, run on real hardware—no emulated latencies. Experiments run on a server with an NVIDIA RTX PRO 6000 (Blackwell) GPU; the GPU-offload path performs AES OpenSSL Project [2024] on its own CUDA stream with completion polled by `cudaEventQuery` NVIDIA Corporation [2024], with the block size sweeping from 4 KiB (light, launch-bound) to 8 MiB (heavy, bandwidth-bound at ∼2.3 ms). For the latency-bound, *remote* offload class (HSM signatures, post-quantum KEMs, remote inference) we stand up a real remote signer over TCP that performs a genuine RSA-2048 signature per request. We drive the event-driven servers with standard load generators (`redis-benchmark`, `ab`). The cross-server summary is Figure 8 and the code cost is in §4.

Event-driven servers. For a single event loop, a synchronous offload caps throughput at one request per offload latency, and the few-line async fix removes the cap. All numbers below use a real GPU AES offload on 1 MiB blocks—realistic bulk crypto (a proxy or server doing TLS on a large transfer)—measured on a genuinely idle GPU. The single-event-loop servers Redis and Node.js gain 3.0× and 2.5×; the event-loop-with-pool servers nginx and memcached gain 2.7× and 2.9×, from an edit of a few dozen lines with no failed requests. A thread/goroutine pool overlaps the offload *automatically*, with no asynchronous code: Apache and a Go server

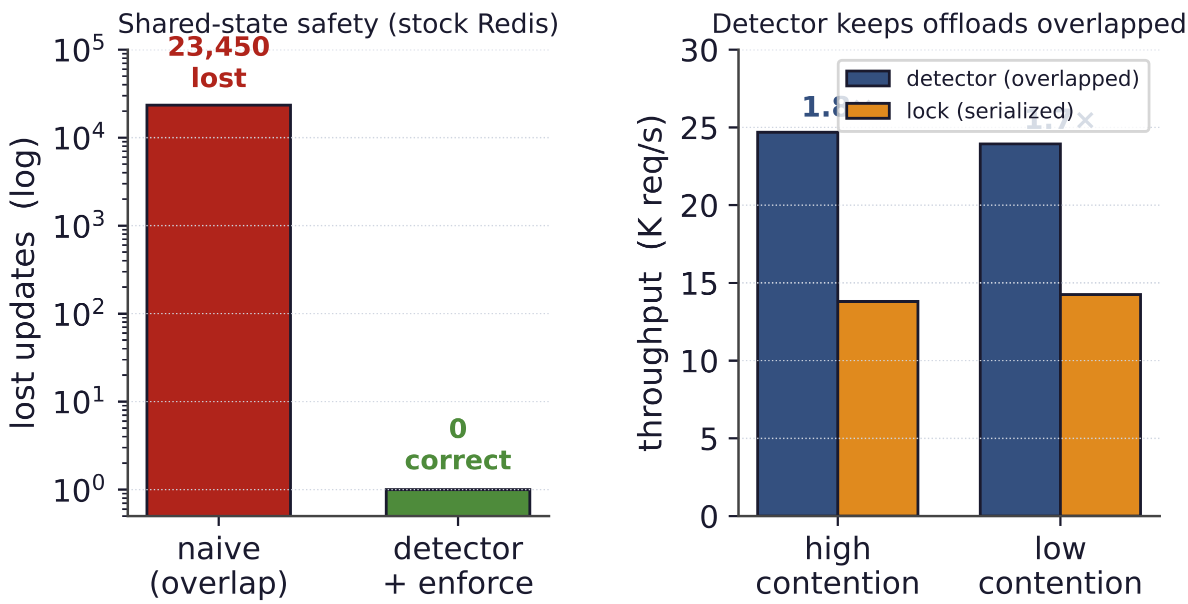

Figure 11. Keeping overlapped stateful offloads correct, measured on a stock Redis server with a real GPU offload. Left: unprotected overlap of a shared read-modify-write loses 23,450 updates; the page-protection detector flags 26,185 conflicts and, under enforcement, reduces lost updates to zero. Right: a coarse lock serializes the very offloads it protects (13.8K req/s), while the detector keeps non-conflicting offloads overlapped (24.7K req/s), 1.8× faster.

gain 3.5× and 3.0× as the runtime runs other workers while one blocks in the offload. A proxy offloads through an external agent over HAProxy's SPOE engine, and here the *agent* must not be the bottleneck: a GIL-bound Python agent gains nothing (0.9×—its per-request protocol handling serializes), but a from-scratch C agent (a real `pthread` worker pool) recovers 2.1× on the same offload. The minimal-edit speedups cluster at 2–3.5× because a real GPU AES offload at this size is bandwidth-bound: overlap fills the device, but cannot exceed its throughput (at 2.1× the proxy's GPU still has headroom—the agent's protocol cost, not the device, caps it).

Latency, not just throughput. Overlap helps latency, not only peak throughput: by keeping the CPU busy during offloads rather than blocking, it sustains low latency to a higher offered load before the latency knee. Under an open-loop arrival process with Poisson arrivals, the overlapping path holds both its median and its tail (p99) latency low to roughly *four* times the offered load that the blocking path can—a median knee near 410 K req/s versus 103 K for blocking (Figure 12)—the operating point that matters for a latency-sensitive service.

Offload weight on a real GPU. Sweeping the real GPU AES offload by block size (consecutive powers of two, 4 KiB to 8 MiB, realistic bulk crypto) shows the predicted weight dependence on a single-event-loop server (Figure 10; full values in Appendix A). The async fix yields essentially no benefit for a light block (1.24× at 4 KiB, 46 µs, on the order of the server's per-request overhead) and grows monotonically to 5.41× at 8 MiB (2.3 ms). The ceiling is a property of the *device*: as the block grows the async path drives the GPU toward its memory bandwidth, so overlap recovers the device *utilization* that blocking wastes—keeping the GPU busy while the synchronous path idles it between requests—but it cannot exceed the device's

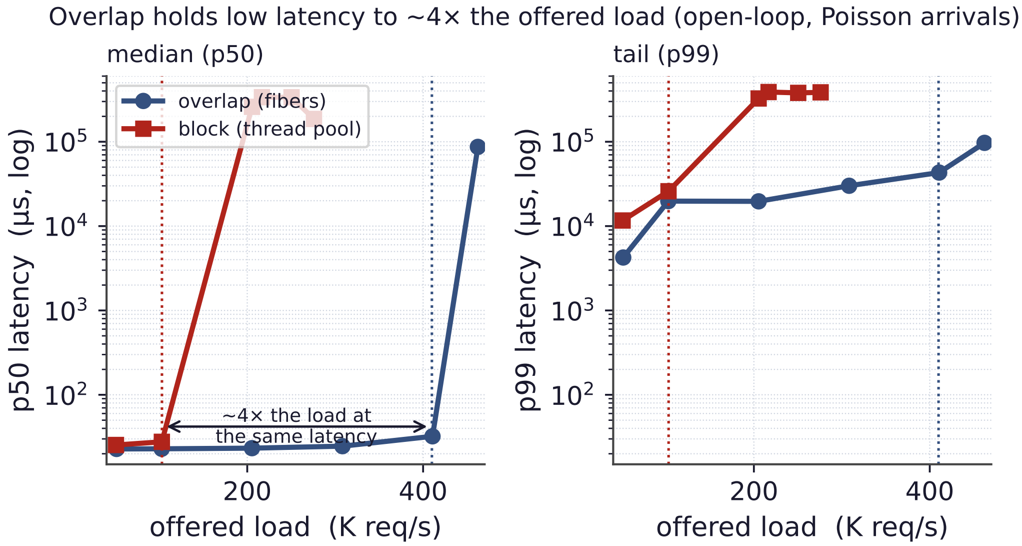

Figure 12. Latency under load (measured, open-loop Poisson arrivals; `runtime/openloop.csv`). Blocking on the offload drives both median (p50, left) and tail (p99, right) latency up at its saturation point near 103 K req/s, while overlapping holds low latency to roughly 4× that offered load (knee near 410 K req/s) before its own. Overlap is a latency win, not only a throughput win.

throughput (a compute-bound kernel would saturate even sooner). A *latency-bound* offload—whose device is not saturated, so many requests can be in flight at once—lifts that device ceiling: we build a real remote signer over TCP (a genuine RSA-2048 signature per request, 821 µs round-trip) and the same Python server reaches 3.53× (2038 → 7202 req/s). Notably this is *not* the order-of-magnitude win a deep device queue could in principle give: the bottleneck has simply moved from the device to the *server's own overlap capacity*—the `asyncio` executor keeps only ∼7 requests genuinely in flight under the GIL. Real overlap on a stock server is thus bounded at both ends, by the accelerator's throughput *and* by how much concurrency the server can actually express; across our servers and offloads it lands at 1.2–5× (and 17.3× only for the purpose-built transparent fiber runtime).

Databases on a real GPU. A process- or thread-per-connection database (PostgreSQL, MariaDB) already overlaps offloads *across* connections for free—the OS runs the backends concurrently, so the GPU is already fed—and the only edit-addressable win is *within* a single query that pipelines many offloads. With a real GPU AES offload this intra-query pipelining is modest—PostgreSQL 1.28× and MariaDB 1.9× through a ∼30-line user-defined function with no core edits—precisely because the launch-bound GPU op is light: once the device is fed, pipelining more of them inside one query cannot exceed its throughput. The same edit, the same GPU, the same per-row work; the win is set by the device, exactly as the model predicts.

Predicted vs. measured. The regime and offload weight predict the outcome: a synchronous offload on a single event loop is catastrophic and the few-line fix recovers it up to the device's throughput limit; the pooled servers overlap automatically with no async code; and the light-offload cases show the predicted absence of benefit. The model of §5 therefore not only summarizes the data but would have told us, in advance, what to expect from each server.

8 Discussion and Limitations

The walls are fundamental. The three walls of §3 are properties of where an application places behavior, not gaps in our engineering. Synchronization issued as a raw system call, thread identity kept in a hardware register, and a workload with no idle CPU are each *below* or *around* the symbol layer a preloaded library can touch, and they generalize: any storage engine with its own futexes, any managed runtime (Go, .NET, V8), and any CPU-bound handler will hit them. This is precisely why the minimal-edit path exists.

When overlap does not pay. Overlap reclaims idle CPU during the offload; when per-request CPU rivals the offload, there is little to reclaim, and the technique should not be applied. The same condition bounds both ends of the spectrum (Wall 3 and Figure 10). For databases, the intra-query win is further bounded by the accelerator's queue depth.

Detector scope. The conflict detector catches read-modify-write conflicts on the monitored data segments; it is a targeted safety net for the overlap hazard, not a general transactional memory. Workloads whose shared state lives outside those segments, or whose correctness depends on properties beyond last-writer-wins, would need stronger machinery.

Complementarity. The two ends of the spectrum are not competitors but a division of labor. Transparent fiberization serves thread-per-connection servers with no source access; the minimal-edit recipe serves everything else, and serves the event-driven servers—the bulk of modern serving—best of all. Together they cover the landscape.

9 Related Work

Our work sits at an under-occupied point in the design space (Figure 13): low modification to *existing* servers *and* broad coverage across server types. Prior approaches achieve one or the other—transparent threading libraries require writing the application in their style, point offload integrations target a single server, and async frameworks assume a rewrite—but not both.

Threads, events, and coroutines. Hiding I/O latency cheaply is an old debate. The event-driven camp (Flash Pai et al. [1999], SEDA Welsh et al. [2001]) avoids thread overhead by structuring the server as a state machine over an event loop, at the cost of "stack ripping"; the user-level-thread camp (Capriccio von Behren et al. [2003], State Threads State Threads Library [2009], `libtask` Cox [2005], core-aware Arachne Qin et al. [2018], and language-level lightweight threads such as Go's goroutines The Go Authors [2024] and Java's virtual threads OpenJDK [2023]) keeps the synchronous coding style and yields at blocking points. Our fiber runtime is in the second lineage and uses a fast register-only switch Li et al. [2023]. But our contribution is not another threading library: it is to *overlap an accelerator offload* in servers built either way, to *quantify the modification cost* of doing so across real servers, and to map where the transparent form provably stops.

Microsecond-scale systems. A body of work rebuilds the OS or runtime to schedule tasks at microsecond timescales efficiently—dataplane operating systems (IX Belay et al. [2014]),

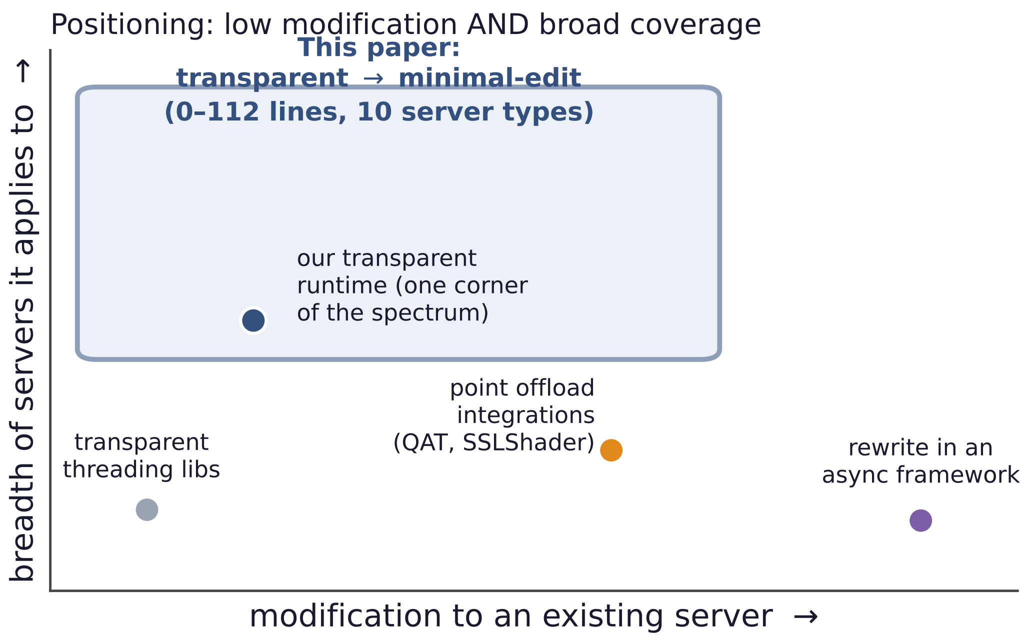

Figure 13. Positioning. Existing approaches to overlapping work are either low-effort but narrow (threading libraries; our own transparent runtime is one such corner) or broad but high-effort (point integrations, async-framework rewrites). The transparent-to-minimal-edit spectrum reaches the desirable region: little modification to existing servers, across many server types.

work-stealing for tail latency (ZygOS Prekas et al. [2017]), and core-reallocating runtimes (Shenango Ousterhout et al. [2019], Caladan Fried et al. [2020], Demikernel Zhang et al. [2021])—typically over kernel-bypass I/O (DPDK DPDK Project [2024], `io_uring` Axboe [2019]). They attack the same killer-microsecond problem Barroso et al. [2017], but demand a new software stack and rewritten applications. We instead take existing, unmodified servers and ask how little it costs to overlap a single offload inside them; the two directions are complementary.

Full hardware offload. One line of work moves the *entire* datapath onto hardware: KV-Direct Li et al. [2017] runs a key-value store in the NIC, ClickNP Li et al. [2016] compiles network functions to an FPGA, iPipe Liu et al. [2018] offloads distributed application logic onto programmable SmartNICs, and GPUnet Kim et al. [2014] lets GPU code drive the network directly. These eliminate the CPU's role rather than overlapping with it, and they require rebuilding the application around the accelerator; iPipe, notably, must itself schedule the offloaded tasks' varying granularity on the NIC—the dual of the latency-hiding problem we solve on the CPU. Our target is the opposite and far more common situation: the application stays on the CPU, the accelerator handles one fine-grained step, and the CPU must hide that step's latency with almost no change to the server.

Accelerator offload integrations. Closer to us, specific systems overlap specific offloads—SSLShader and GPU-crypto work overlapped TLS Jang et al. [2011], QAT engines Intel Corporation [2024] plug compression and crypto into nginx's asynchronous paths, and inference-serving systems such as RedisAI RedisAI [2022] and Clipper Crankshaw et al. [2017] batch and pipeline GPU work. These are valuable point integrations, each tuned to one server and one offload. We instead give a *predictive model* that explains and anticipates the win across the server landscape, a uniform recipe, and the transparent boundary.

Asynchronous frameworks. Event and async frameworks (libevent Provos and Mathewson [2024], libuv libuv contributors [2024], Seastar ScyllaDB [2024], `async/await`) make overlap the default—if one writes the application in them from the start. Our concern is the opposite: injecting overlap into *existing* servers with minimal change. Proxy offload engines (HAProxy's SPOE HAProxy Technologies [2024b], Envoy's `ext_proc` Envoy Project [2024]) make async offload a first-class, configuration-only feature for the proxy tier; we measure the HAProxy case directly.

Conflict detection and speculation. Page-protection dirty-tracking, software distributed shared memory Amza et al. [1996], and transactional memory Herlihy and Moss [1993] all detect or prevent conflicting concurrent writes. Our detector borrows the page-protection technique but is deliberately lightweight and specialized to the one hazard overlap introduces—a read-modify-write reordered across an offload.

Symbol interposition. The fragility of interposing versioned glibc symbols is folklore among practitioners; our condition-variable finding documents a concrete, reproducible instance and its fix.

10 Conclusion

What should the CPU do while it waits for the accelerator? It should *overlap*: do other useful work while the offload is in flight. The real question is how much an existing server must change to make that happen, and the answer is a spectrum from zero lines to about a hundred. At the zero-edit end, a transparent runtime overlaps an unmodified thread-per-connection binary—but it reaches a narrow niche and hits three architectural walls, and it cannot help the event-driven servers that have the most to gain. A few lines at the offload site, routed through a server's own concurrency mechanism, work everywhere; measured on real hardware they recover 1.2–5× across ten production servers. The win is bounded at both ends—by the accelerator's own throughput (a large GPU offload saturates the device's bandwidth) and by the server's own overlap capacity (even a real latency-bound remote signer tops out at 3.5× behind a GIL-bound executor)—so the order-of-magnitude wins an unbounded offload model predicts do not survive real hardware, except in the purpose-built transparent runtime (17.3×). A simple model—concurrency model crossed with offload weight—predicts both the speedup and the code cost in advance, and a transparent conflict detector, validated on a stock Redis server, keeps the overlap correct when it touches shared state. The result is a practical recipe, and a map, for hiding fine-grained accelerator-offload latency in the servers that run online services today.

A AES Block-Size Sweep (detailed values)

Table 1 gives the full per-size measurements behind the offload-weight curve of Figure 10, all on a real GPU on an *idle* device (no co-tenant compute), driving a single-event-loop server

(Python `asyncio` with a 32-thread executor) at 50 concurrent clients. The AES block size is swept over consecutive powers of two from 4 KiB to 8 MiB; for each we report the measured single-op GPU latency (one offload in flight), the synchronous and asynchronous throughput, and their ratio. The speedup grows monotonically from 1.24× at 4 KiB (launch-bound: the offload is tens of microseconds, on the order of the server's own per-request work, so there is little to overlap) to 5.41× at 8 MiB (the offload is bandwidth-bound at ∼2.3 ms, so overlapping it across requests reclaims most of the wait), up to the GPU's throughput ceiling.

block size	GPU latency (µs)	sync (req/s)	async (req/s)	speedup
4 KiB	46.4	3750	4667	1.24×
8 KiB	47.0	3688	4650	1.26×
16 KiB	51.4	3657	4573	1.25×
32 KiB	60.3	3501	4371	1.25×
64 KiB	70.2	3326	4355	1.31×
128 KiB	92.7	2970	4218	1.42×
256 KiB	126.5	2706	4128	1.53×
512 KiB	191.8	2454	4494	1.83×
1 MiB	361.6	1538	3644	2.37×
2 MiB	653.6	966	2858	2.96×
4 MiB	1188.1	506	1937	3.83×
8 MiB	2258.6	227	1228	5.41×

Table 1. Real-GPU AES block-size sweep on a single-event-loop server (idle GPU). Source: `transparent-runtime/apps/aes_blocksize_py_results.csv`.

References

Cristiana Amza, Alan L. Cox, Sandhya Dwarkadas, Pete Keleher, Honghui Lu, Ramakrishnan Rajamony, Weimin Yu, and Willy Zwaenepoel. TreadMarks: Shared memory computing on networks of workstations. *IEEE Computer*, 29(2):18–28, 1996.

Apache Software Foundation. Apache HTTP server. https://httpd.apache.org/, 2024.

Jens Axboe. Efficient IO with io_uring. https://kernel.dk/io_uring.pdf, 2019.

Luiz Barroso, Mike Marty, David Patterson, and Parthasarathy Ranganathan. Attack of the killer microseconds. *Communications of the ACM*, 60(4):48–54, 2017.

Adam Belay, George Prekas, Ana Klimovic, Samuel Grossman, Christos Kozyrakis, and Edouard Bugnion. IX: A protected dataplane operating system for high throughput and low latency. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 49–65, 2014.

Russ Cox. libtask: A coroutine library for C and Unix. https://swtch.com/libtask/, 2005.

Daniel Crankshaw, Xin Wang, Giulio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *Proceedings of the 14th*

USENIX Symposium on Networked Systems Design and Implementation (NSDI), pages 613–627, 2017.

DPDK Project. DPDK: Data plane development kit. https://www.dpdk.org/, 2024.

Ulrich Drepper. How to write shared libraries. https://www.akkadia.org/drepper/dsohowto.pdf, 2011.

Envoy Project. External processing filter (ext_proc). https://www.envoyproxy.io/docs/envoy/latest/configuration/http/http_filters/ext_proc_filter, 2024.

F5/NGINX. nginx: Http and reverse proxy server. https://nginx.org/, 2024.

Hubertus Franke, Rusty Russell, and Matthew Kirkwood. Fuss, futexes and furwocks: Fast userlevel locking in Linux. In *Proceedings of the Ottawa Linux Symposium (OLS)*, pages 479–495, 2002.

Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. Caladan: Mitigating interference at microsecond timescales. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 281–297, 2020.

HAProxy Technologies. HAProxy: The reliable, high-performance TCP/HTTP load balancer. https://www.haproxy.org/, 2024a.

HAProxy Technologies. Stream processing offload engine (SPOE) and protocol (SPOP). https://www.haproxy.com/documentation/, 2024b.

Maurice Herlihy and J. Eliot B. Moss. Transactional memory: Architectural support for lock-free data structures. In *Proceedings of the 20th Annual International Symposium on Computer Architecture (ISCA)*, pages 289–300, 1993.

Intel Corporation. Intel quickassist technology (Intel QAT). https://www.intel.com/content/www/us/en/architecture-and-technology/intel-quick-assist-technology-overview.html, 2024.

Keon Jang, Sangjin Han, Seungyeop Han, Sue Moon, and KyoungSoo Park. SSLShader: Cheap SSL acceleration with commodity processors. In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2011.

Sangman Kim, Seonggu Huh, Xinya Zhang, Yige Hu, Amir Wated, Emmett Witchel, and Mark Silberstein. GPUnet: Networking abstractions for GPU programs. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 201–216, 2014.

Bojie Li, Kun Tan, Layong (Larry) Luo, Yanqing Peng, Renqian Luo, Ningyi Xu, Yongqiang Xiong, Peng Cheng, and Enhong Chen. ClickNP: Highly flexible and high performance network processing with reconfigurable hardware. In *Proceedings of the 2016 ACM SIGCOMM Conference*, pages 1–14, 2016.

Bojie Li, Zhenyuan Ruan, Wencong Xiao, Yuanwei Lu, Yongqiang Xiong, Andrew Putnam, Enhong Chen, and Lintao Zhang. KV-Direct: High-performance in-memory key-value store with programmable NIC. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 137–152, 2017.

Bojie Li, Zihao Xiang, Xiaoliang Wang, Han Ruan, Jingbin Zhou, and Kun Tan. FastWake: Revisiting host network stack for interrupt-mode RDMA. In *Proceedings of the 7th Asia-Pacific Workshop on Networking (APNet)*, pages 1–7. ACM, 2023. doi: 10.1145/3600061.3600063.

libuv contributors. libuv: Cross-platform asynchronous I/O. https://libuv.org/, 2024.

Ming Liu, Tianyi Cui, Henry Schuh, Arvind Krishnamurthy, Simon Peter, and Karan Gupta. Offloading distributed applications onto SmartNICs using iPipe. In *Proceedings of the 2018 ACM SIGCOMM Conference*, pages 318–333, 2018.

MariaDB Foundation. MariaDB server: The open source relational database. https://mariadb.org/, 2024.

memcached. memcached: A distributed memory object caching system. https://memcached.org/, 2024.

National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Technical Report FIPS 203, NIST, 2024.

NVIDIA Corporation. CUDA C++ programming guide: Asynchronous concurrent execution. https://docs.nvidia.com/cuda/cuda-c-programming-guide/, 2024.

OpenJDK. JEP 444: Virtual threads (project Loom). https://openjdk.org/jeps/444, 2023.

OpenJS Foundation. Node.js: A JavaScript runtime built on V8. https://nodejs.org/, 2024.

OpenSSL Project. OpenSSL: Cryptography and SSL/TLS toolkit. https://www.openssl.org/, 2024.

Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. Shenango: Achieving high CPU efficiency for latency-sensitive datacenter workloads. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 361–378, 2019.

Vivek S. Pai, Peter Druschel, and Willy Zwaenepoel. Flash: An efficient and portable web server. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, 1999.

PostgreSQL Global Development Group. PostgreSQL: The world's most advanced open source relational database. https://www.postgresql.org/, 2024.

George Prekas, Marios Kogias, and Edouard Bugnion. ZygOS: Achieving low tail latency for microsecond-scale networked tasks. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 325–341, 2017.

Niels Provos and Nick Mathewson. libevent: An event notification library. https://libevent.org/, 2024.

Python Software Foundation. asyncio: Asynchronous I/O in CPython. `https://docs.python.org/3/library/asyncio.html`, 2024.

Henry Qin, Qian Li, Jacqueline Speiser, Peter Kraft, and John Ousterhout. Arachne: Core-aware thread management. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 145–160, 2018.

Redis Ltd. Redis: An in-memory data store. `https://redis.io/`, 2024.

RedisAI. RedisAI: A redis module for serving tensors and executing deep learning models. `https://oss.redis.com/redisai/`, 2022.

ScyllaDB. Seastar: A high-performance shared-nothing asynchronous C++ framework. In *`https://seastar.io/`*, 2024.

State Threads Library. State threads library for internet applications. `http://state-threads.sourceforge.net/`, 2009.

The Go Authors. The Go programming language: Goroutines and the scheduler. `https://go.dev/`, 2024.

Rob von Behren, Jeremy Condit, Feng Zhou, George C. Necula, and Eric Brewer. Capriccio: Scalable threads for internet services. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*, pages 268–281, 2003.

Matt Welsh, David Culler, and Eric Brewer. SEDA: An architecture for well-conditioned, scalable internet services. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*, pages 230–243, 2001.

Irene Zhang, Amanda Raybuck, Pratyush Patel, Kirk Olynyk, Jacob Nelson, Omar S. Navarro Leija, Ashlie Martinez, Jing Liu, Anna Kornfeld Simpson, Sujay Jayakar, Pedro Henrique Penna, Max Demoulin, Piali Choudhury, and Anirudh Badam. The demikernel datapath OS architecture for microsecond-scale datacenter systems. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, pages 195–211, 2021.